

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN – ĐHQG-HCM
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG



ĐỒ ÁN MÔN LẬP TRÌNH ỨNG DỤNG WEB

**URL SHORTENER - TRANG WEB RÚT GỌN
LIÊN KẾT VÀ PHÂN TÍCH LƯỢT TRUY CẬP**

Môn học:	Lập trình ứng dụng Web
Lớp:	NT208.Q23.ANTT
Giảng viên hướng dẫn:	Nghi Hoàng Khoa
Nhóm thực hiện:	Nhóm 16
Sinh viên thực hiện:	Nguyễn Vũ Đức Hạnh - 24520453

TP. Hồ Chí Minh, tháng 5 năm 2026

LỜI CẢM ƠN

Kính gửi Thầy Nghi Hoàng Khoa,

Lời đầu tiên, em xin chân thành cảm ơn Thầy vì đã truyền đạt những kiến thức, kinh nghiệm thực tế quý báu và hướng dẫn tận tình cho chúng em nhiệt tình trong môn học Lập trình ứng dụng Web.

Qua những buổi học cùng với sự hướng dẫn tận tình của Thầy, em đã nắm được luồng hoạt động của một trang web, cũng như học được các kiến thức và kỹ năng để triển khai một trang web từ Frontend đến Backend, cách triển khai ra thực tế một trang web hiệu quả. Đồng thời thông qua các bài tập, em cũng nắm được các lỗi thường gặp khi triển khai web cũng như cách tìm và xử lý bug trong một dự án, biết cách áp dụng những kiến thức được học trong các trường hợp thực tế và trong quá trình thực hiện đồ án.

Em cũng xin bày tỏ lòng biết ơn đối với những lời khuyên quý giá từ Thầy trong quá trình học tập và làm đồ án. Những kiến thức đã được học ở môn học này sẽ là tiền đề quan trọng cho chúng em áp dụng vào công việc và cuộc sống sau này.

Lời cuối cùng, vì giới hạn trong kiến thức chuyên ngành cũng như kinh nghiệm thực tế, đồ án môn học Lập trình ứng dụng Web của em không tránh khỏi những sai sót và hạn chế trong quá trình thực hiện. Em rất mong sẽ nhận được những nhận xét, góp ý chuyên môn, tận tình từ Thầy để có cơ hội hoàn thiện đồ án môn học này một cách tốt nhất.

Em xin chân thành cảm ơn!

TP. Hồ Chí Minh, tháng 6 năm 2026

Sinh viên thực hiện

Nguyễn Vũ Đức Hạnh

TÓM TẮT ĐỒ ÁN

Đồ án “**URL Shortener - Trang web rút gọn liên kết và phân tích lượt truy cập**” được xây dựng nhằm giải quyết nhu cầu tối ưu hóa và quản lý các đường dẫn dài trên Internet. Hệ thống được phát triển theo mô hình Monolith sử dụng Node.js (Express.js) cho phía máy chủ, kết hợp với cơ sở dữ liệu MySQL thông qua Prisma ORM và Redis để tối ưu hóa tốc độ chuyển hướng (Cache).

Các tính năng chính bao gồm: rút gọn URL bằng thuật toán mã hóa Base62, tạo các liên kết tùy chỉnh (custom alias), thiết lập thời hạn cho liên kết, quản lý tài khoản người dùng an toàn với mã hóa Bcrypt, và theo dõi chi tiết các số liệu thống kê trực quan (lượt click theo ngày, nguồn referer, quốc gia truy cập) bằng thư viện Chart.js.

Đồng thời, em cũng đã thực hiện triển khai lên máy chủ ảo Ubuntu (VPS DigitalOcean) thông qua công nghệ Container hóa Docker. Hệ thống được thiết lập các tiêu chuẩn an toàn: quản trị cơ sở dữ liệu bảo mật qua SSH Tunneling, phân giải tên miền thực tế **sht-url.me**, định tuyến bằng Nginx Reverse Proxy, và nâng cấp toàn bộ luồng truyền tải dữ liệu lên giao thức mã hóa HTTPS với chứng chỉ Let's Encrypt.

Kết quả đạt được sau khi hoàn thành đồ án: hệ thống hoạt động ổn định trên môi trường Production, thời gian phản hồi redirect dưới 5ms trong trường hợp cache hit, và cung cấp giao diện quản trị thân thiện cho người dùng cuối.

Mục lục

Lời cảm ơn	i
Tóm tắt đồ án	ii
Danh mục hình ảnh	vii
Danh mục bảng	viii
Danh mục từ viết tắt	ix
1 Tổng quan dự án	1
1.1 Đặt vấn đề	1
1.2 Mục tiêu đồ án	1
1.3 Phạm vi đồ án	1
1.3.1 Phạm vi chức năng	1
1.3.2 Phạm vi kỹ thuật	2
1.3.3 Giới hạn của đồ án	2
1.4 Đối tượng người dùng	2
2 Tổng quan Technology Stack	3
3 Kiến trúc hệ thống	4
3.1 Mô hình kiến trúc tổng thể	4
3.2 Phân tích các tầng kiến trúc	5
3.2.1 Tầng Client (Presentation Layer)	5
3.2.2 Tầng Reverse Proxy (Production)	5
3.2.3 Tầng Application (Business Logic Layer)	5
3.2.4 Tầng Data (Persistence Layer)	5
4 Cơ sở dữ liệu	6
4.1 Thiết kế CSDL	6
4.1.1 Bảng <code>users</code>	6
4.1.2 Bảng <code>urls</code>	6
4.1.3 Bảng <code>click_events</code>	7
4.2 Mã nguồn Prisma Schema	7
4.3 Quan hệ giữa các bảng	9
4.4 Redis Cache	9

5	Backend	10
5.1	Khởi động server	10
5.2	Session Store tùy chỉnh	11
5.3	Luồng xác thực (Authentication Flow)	12
5.3.1	Luồng đăng ký (Register)	12
5.3.2	Luồng đăng nhập (Login)	13
5.3.3	Middleware bảo vệ route	13
5.4	URL Shortening - Logic hoạt động	14
5.4.1	Trường hợp 1: Auto-generate shortCode (Base62)	14
5.4.2	Trường hợp 2: Custom Alias	14
5.4.3	Thuật toán Base62 Encoding	14
5.5	Redirect - Cache-First Logic	15
5.6	Analytics - Thu thập và tổng hợp	16
5.6.1	Ghi Click Event	16
5.6.2	Tổng hợp Analytics 30 ngày	16
5.7	Dashboard - Quản lý URL	18
5.7.1	Luồng xóa URL	18
5.8	Middleware bảo vệ hệ thống	18
5.8.1	Rate Limiter	18
5.8.2	Error Handler toàn cục	19
5.9	Health Check Endpoint	19
6	Frontend	20
6.1	Các trang giao diện	20
6.2	Giao tiếp API - File <code>api.js</code>	20
6.3	Xác thực và đồng bộ Navbar - <code>auth.js</code>	21
6.4	Form rút gọn URL - <code>shorten.js</code>	21
6.5	Dashboard - <code>dashboard.js</code>	22
6.6	Trang Analytics - <code>analytics.js</code>	22
6.7	Utility functions - <code>utils.js</code>	22
7	API Endpoints	23
7.1	Bảng tổng hợp API	23
7.2	Chi tiết Request/Response các API chính	24
7.2.1	POST <code>/api/auth/register</code>	24
7.2.2	POST <code>/api/shorten</code>	24
7.2.3	GET <code>/:shortCode</code> (Redirect)	25
7.2.4	GET <code>/api/urls/:id/analytics</code>	25
7.3	HTTP Status Code	26

8	Redis	27
8.1	Tổng quan kiến trúc Redis	27
8.2	Tầng 1: Session Store	27
8.2.1	Vai trò	27
8.2.2	Lưu đồ hoạt động	28
8.2.3	Ưu điểm của Redis làm Session Store	28
8.3	Tầng 2: URL Cache	28
8.3.1	Vai trò	28
8.3.2	Cache Service Wrapper	29
8.3.3	Chiến lược Cache Invalidation	29
8.4	So sánh hiệu năng với và không có cache	30
8.5	Cấu hình ioredis	30
9	Đóng gói và triển khai với Docker	30
9.1	Tổng quan Container hóa	30
9.2	Dockerfile của ứng dụng	31
9.3	Docker Compose Configuration	31
9.4	Sơ đồ kiến trúc Docker	34
10	Bảo mật	35
10.1	Tổng quan các biện pháp bảo mật	35
10.2	Chi tiết một số biện pháp quan trọng	35
10.2.1	Password Hashing với Bcrypt	35
10.2.2	Ownership Check (Authorization)	36
10.2.3	XSS Prevention	36
10.2.4	SQL Injection Prevention	37
10.3	Bảo mật cookie	37
11	Triển khai Production và HTTPS	37
11.1	Hạ tầng Máy chủ (VPS)	37
11.2	Quy trình thiết lập Docker và xử lý sự cố	37
11.2.1	Cài đặt môi trường	37
11.2.2	Xử lý lỗi thư viện Prisma (OpenSSL)	38
11.2.3	Cấu hình phục vụ giao diện tĩnh	38
11.3	Quản trị Database an toàn (SSH Tunneling)	38
11.4	Cấu hình tên miền và HTTPS	39
11.4.1	Đăng ký tên miền	39
11.4.2	Xử lý xung đột Port 80	39
11.4.3	Cài đặt SSL với Let's Encrypt	39
11.4.4	Cấu hình Nginx Reverse Proxy	39

11.5 Quy trình deploy	40
12 Kết luận	41
12.1 Kết luận	41
12.2 Hạn chế của đồ án	41
Tài liệu tham khảo	42

Danh mục hình ảnh

1	Sơ đồ kiến trúc tổng thể của hệ thống URL Shortener	4
2	Sơ đồ ERD - Quan hệ giữa các bảng trong CSDL	9
3	Luồng xử lý đăng ký tài khoản	12
4	Luồng xử lý đăng nhập	13
5	Luồng tạo shortCode bằng Base62	14
6	Luồng tạo URL với custom alias	14
7	Luồng xóa URL với kiểm tra quyền sở hữu	18
8	Luồng đồng bộ trạng thái đăng nhập trên Navbar	21
9	Luồng rút gọn URL từ phía client	21
10	Luồng tải Dashboard	22
11	Luồng tải trang Analyticsp	22
12	Luồng hoạt động của Session Store	28
13	Kiến trúc các container Docker	34
14	Sơ đồ SSH Tunneling cho quản trị DB	38

Danh mục bảng

1	Đối tượng người dùng của hệ thống	2
2	Bảng tổng hợp công nghệ sử dụng	3
3	Cấu trúc bảng users	6
4	Cấu trúc bảng urls	6
5	Cấu trúc bảng click_events	7
6	Hai tầng sử dụng Redis trong hệ thống	10
7	Tương quan giữa Auto-increment ID và Base62 shortCode	15
8	Danh sách các trang giao diện	20
9	Bảng tổng hợp các API endpoint của hệ thống	23
10	HTTP Status Code được sử dụng	26
11	Cấu hình Redis trong hệ thống	27
12	Chiến lược quản lý cache	29
13	So sánh latency redirect	30
14	Bảng tổng hợp các biện pháp bảo mật	35

Danh mục từ viết tắt

Từ viết tắt	Ý nghĩa
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DB	Database
DNS	Domain Name System
HTML	HyperText Markup Language
HTTP/HTTPS	HyperText Transfer Protocol / Secure
JSON	JavaScript Object Notation
JWT	JSON Web Token
MVC	Model - View - Controller
ORM	Object - Relational Mapping
REST	Representational State Transfer
SQL	Structured Query Language
SSH	Secure Shell
SSL/TLS	Secure Sockets Layer / Transport Layer Security
TTL	Time To Live
UI/UX	User Interface / User Experience
URL	Uniform Resource Locator
VPS	Virtual Private Server
XSS	Cross-Site Scripting

1. Tổng quan dự án

1.1. Đặt vấn đề

Trong kỷ nguyên số hiện nay, việc chia sẻ thông tin qua các liên kết (URL) trở thành nhu cầu thiết yếu trong giao tiếp số. Tuy nhiên, các URL gốc thường có độ dài lớn, chứa nhiều tham số phức tạp, gây khó khăn cho người dùng khi:

- Chia sẻ qua các nền tảng giới hạn ký tự như Twitter, SMS.
- In ấn trên các tài liệu giấy, danh thiếp, mã QR.
- Ghi nhớ và đọc thủ công.
- Theo dõi hiệu quả của các chiến dịch marketing thông qua từng liên kết.

1.2. Mục tiêu đề án

1. **Mục tiêu chức năng:** Phát triển một hệ thống web hoàn chỉnh cho phép người dùng rút gọn URL, theo dõi lượt truy cập và quản lý các liên kết đã tạo.
2. **Mục tiêu kỹ thuật:** Áp dụng Node.js, Express.js, Prisma ORM, MySQL, Redis theo mô hình kiến trúc phân lớp (Layered Architecture).
3. **Mục tiêu hiệu năng:** Tối ưu thời gian phản hồi redirect dưới 5ms bằng cơ chế cache với Redis.
4. **Mục tiêu bảo mật:** Triển khai các biện pháp bảo mật web cơ bản (hash mật khẩu, session cookie, rate limiting, ownership check, HTTPS).
5. **Mục tiêu triển khai:** Đóng gói toàn bộ hệ thống bằng Docker và triển khai thành công lên môi trường Production thực tế với tên miền và chứng chỉ SSL.

1.3. Phạm vi đề án

1.3.1. Phạm vi chức năng

Hệ thống cung cấp các nhóm chức năng chính:

- **Rút gọn URL:** Tạo liên kết ngắn từ URL gốc bằng thuật toán Base62 hoặc custom alias do người dùng chỉ định.
- **Redirect tốc độ cao:** Chuyển hướng từ URL ngắn sang URL gốc với độ trễ thấp nhờ Redis cache.
- **Phân tích lượt truy cập:** Thống kê chi tiết theo ngày, theo nguồn truy cập (referer), theo quốc gia (qua GeoIP lookup), hiển thị trực quan bằng biểu đồ.
- **Quản lý tài khoản:** Đăng ký, đăng nhập, đăng xuất với cơ chế session cookie.
- **Dashboard quản lý:** Giao diện liệt kê, sao chép, xóa các liên kết của người dùng.
- **Tính năng nâng cao:** Custom alias, expiration date, rate limiting, health check endpoint.

1.3.2. Phạm vi kỹ thuật

- **Mô hình kiến trúc:** Full-stack monolith với Backend Node.js và Frontend HTML/CSS/JS thuần.
- **Cơ sở dữ liệu:** MySQL 8.0 cho dữ liệu persistent, Redis 7 cho cache và session store.
- **Triển khai:** Container hóa bằng Docker Compose, deploy trên VPS DigitalOcean với Nginx Reverse Proxy và HTTPS.

1.3.3. Giới hạn của đồ án

- Chưa hỗ trợ tính năng OAuth (đăng nhập qua Google, Facebook).
- Chưa có cơ chế phân quyền nâng cao (chỉ có vai trò người dùng thông thường, không có admin role).
- Chưa tích hợp CI/CD pipeline tự động.

1.4. Đối tượng người dùng

Bảng 1: Đối tượng người dùng của hệ thống

Đối tượng	Nhu cầu sử dụng
Người dùng khách (Guest)	Rút gọn URL nhanh, không cần đăng ký tài khoản.
Người dùng đăng ký	Quản lý danh sách URL đã tạo, xem thống kê chi tiết, cài đặt thời hạn cho liên kết.
Nhà tiếp thị (Marketer)	Theo dõi hiệu quả chiến dịch qua phân tích referer và lượt truy cập theo thời gian.
Lập trình viên	Tích hợp API rút gọn URL vào ứng dụng khác.

2. Tổng quan Technology Stack

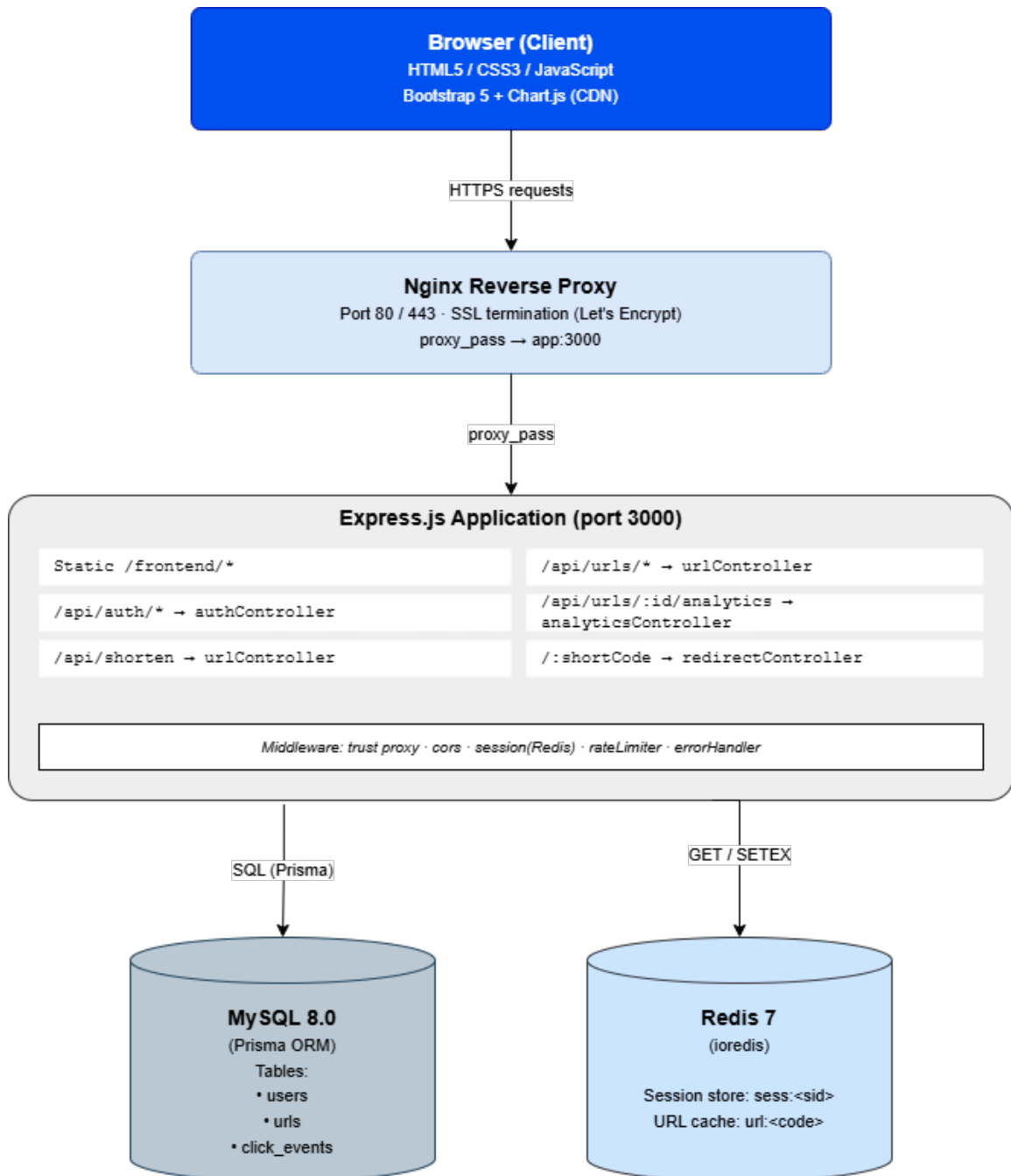
Bảng 2: Bảng tổng hợp công nghệ sử dụng

Tầng	Công nghệ	Phiên bản	Vai trò
Runtime	Node.js	18 LTS	Môi trường JavaScript server-side
Framework	Express.js	4.x	HTTP server, routing, middleware
ORM	Prisma	5.x	Truy vấn DB, migration, schema management
Database	MySQL	8.0	Lưu trữ dữ liệu persistent
Cache	Redis (ioredis)	7	Session store và URL cache
Authentication	express-session, bcryptjs	-	Quản lý phiên + hash mật khẩu
GeoIP	geoip-lite	1.4.x	Tra cứu quốc gia từ địa chỉ IP (offline DB)
Frontend	HTML5, CSS3, Bootstrap 5	-	Static UI
Charts	Chart.js	4 (CDN)	Hiển thị biểu đồ analytics
Testing	Jest, Supertest	-	Unit và integration test
Load Test	k6	-	Performance benchmark
Container	Docker, Docker Compose	-	Đóng gói triển khai
Reverse Proxy	Nginx (alpine)	-	Proxy HTTPS trên Production
SSL/TLS	Let's Encrypt (Certbot)	-	Chứng chỉ SSL miễn phí

3. Kiến trúc hệ thống

3.1. Mô hình kiến trúc tổng thể

Hệ thống được thiết kế theo mô hình **Layered Monolith Architecture** (Kiến trúc nguyên khối phân lớp), với các tầng được phân tách rõ ràng. Trên môi trường Production, kiến trúc được mở rộng thêm tầng Nginx Reverse Proxy ở phía trước để xử lý HTTPS và phân phối tải.



Hình 1: Sơ đồ kiến trúc tổng thể của hệ thống URL Shortener

3.2. Phân tích các tầng kiến trúc

3.2.1. Tầng Client (Presentation Layer)

- Bao gồm các trang HTML tĩnh: `index.html`, `signin.html`, `register.html`, `dashboard.html`, `analytics.html`.
- Sử dụng Bootstrap 5 cho responsive layout và Chart.js cho biểu đồ.
- Giao tiếp với server qua RESTful API bằng `fetch()` API, gửi kèm session cookie qua `credentials: 'include'`.

3.2.2. Tầng Reverse Proxy (Production)

- Nginx lắng nghe trên cổng 80/443 (HTTPS), nhận tất cả request từ Internet.
- Thực hiện **SSL termination**: giải mã TLS, chuyển tiếp HTTP plain text đến container app.
- Cung cấp các header bảo mật: `X-Forwarded-For`, `X-Real-IP` để Express lấy IP thực.

3.2.3. Tầng Application (Business Logic Layer)

Express.js xử lý request theo pipeline middleware:

1. `trust proxy` - tin tưởng header từ Nginx.
2. `cors` - xử lý cross-origin với `credentials: true`.
3. `express.json()` - parse body JSON.
4. `cookieParser()` - parse cookie.
5. `session` - quản lý phiên với Redis store.
6. Routes - phân phối theo prefix endpoint.
7. `errorHandler` - bắt lỗi toàn cục.

Trong Application Layer, mã nguồn được tổ chức theo pattern **Controller-Service-Utility**:

- **Controllers**: Chỉ xử lý HTTP (parse req, gọi service, trả response).
- **Services**: Chứa business logic, gọi Prisma và Redis.
- **Utilities**: Pure functions không có side-effect (`base62`, `validator`).

3.2.4. Tầng Data (Persistence Layer)

- **MySQL 8.0**: Lưu trữ persistent với 3 bảng (`users`, `urls`, `click_events`).
- **Redis 7**: Cache 2 mục đích:
 - *URL cache*: key = `shortCode`, value = `longUrl`, TTL 1 ngày.
 - *Session store*: key = `sess:<sid>`, value = JSON session, TTL 7 ngày.

4. Cơ sở dữ liệu

4.1. Thiết kế CSDL

Cơ sở dữ liệu được thiết kế gồm 3 bảng chính, quản lý bởi Prisma ORM thông qua file `schema.prisma`.

4.1.1. Bảng users

Bảng 3: Cấu trúc bảng users

Trường	Kiểu	Ràng buộc	Mô tả
id	INT	PK, AUTO_INCREMENT	Khóa chính
email	VARCHAR(255)	UNIQUE, NOT NULL	Email đăng nhập
password	VARCHAR(255)	NOT NULL	Mật khẩu đã hash bằng bcrypt (10 rounds)
createdAt	DATETIME	DEFAULT NOW()	Thời điểm đăng ký

4.1.2. Bảng urls

Bảng 4: Cấu trúc bảng urls

Trường	Kiểu	Ràng buộc	Mô tả
id	INT	PK, AUTO_INCREMENT	Khóa chính
shortCode	VARCHAR(20)	UNIQUE, NOT NULL	Mã rút gọn (base62 hoặc alias)
longUrl	TEXT	NOT NULL	URL gốc
userId	INT	FK -> users(id), NULL	Chủ sở hữu (null nếu guest)
clicks	INT	DEFAULT 0	Tổng số lượt click
expiresAt	DATETIME	NULL	Thời điểm hết hạn (optional)
createdAt	DATETIME	DEFAULT NOW()	Thời điểm tạo

Index: INDEX(userId) để truy vấn nhanh danh sách URL của một user.

4.1.3. Bảng click_events

Bảng 5: Cấu trúc bảng click_events

Trường	Kiểu	Ràng buộc	Mô tả
id	INT	PK, AUTO_INCREMENT	Khóa chính
urlId	INT	FK -> urls(id) CASCADE	URL được click
ipAddress	VARCHAR(45)	NULL	Địa chỉ IPv4 hoặc IPv6
userAgent	TEXT	NULL	User-Agent của browser
referrer	TEXT	NULL	Trang nguồn click
country	VARCHAR(50)	NULL	Mã quốc gia ISO 3166-1 alpha-2, tra cứu qua geoip-lite từ ipAddress
clickedAt	DATETIME	DEFAULT NOW()	Thời điểm click

Index:

- INDEX(urlId) - truy vấn theo URL.
- INDEX(clickedAt) - truy vấn theo thời gian.
- INDEX(urlId, clickedAt) - composite index cho query analytics theo thời gian của 1 URL.

4.2. Mã nguồn Prisma Schema

```

1 datasource db {
2   provider = "mysql"
3   url      = env("DATABASE_URL")
4 }
5
6 generator client {
7   provider = "prisma-client-js"
8 }
9
10 model User {
11   id          Int      @id @default(autoincrement())
12   email       String   @unique
13   password    String   // bcrypt hash (10 rounds)
14   createdAt   DateTime @default(now()) @map("created_at")
15   urls        Url[]
16   @@map("users")

```

```

17 }
18
19 model Url {
20   id          Int          @id @default(autoincrement())
21   shortCode   String       @unique @map("short_code")
22   longUrl     String       @map("long_url") @db.Text
23   userId      Int?         @map("user_id")
24   clicks      Int          @default(0)
25   expiresAt   DateTime?    @map("expires_at")
26   createdAt   DateTime     @default(now()) @map("created_at")
27   user        User?        @relation(fields: [userId], references: [id]
28   ],
29                                   onDelete: SetNull)
29   events      ClickEvent[]
30   @@index([userId])
31   @@map("urls")
32 }
33
34 model ClickEvent {
35   id          Int          @id @default(autoincrement())
36   urlId       Int          @map("url_id")
37   ipAddress   String?      @map("ip_address") @db.VarChar(45)
38   userAgent   String?      @map("user_agent") @db.Text
39   referer     String?      @db.Text
40   country     String?      @db.VarChar(50)
41   clickedAt   DateTime     @default(now()) @map("clicked_at")
42   url         Url          @relation(fields: [urlId], references: [id],
43                                   onDelete: Cascade)
44   @@index([urlId])
45   @@index([clickedAt])
46   @@index([urlId, clickedAt])
47   @@map("click_events")
48 }

```

Listing 1: File backend/prisma/schema.prisma

4.3. Quan hệ giữa các bảng



Hình 2: Sơ đồ ERD - Quan hệ giữa các bảng trong CSDL

Các quan hệ trong CSDL:

- **users (1) - urls (N)**: Một user có thể tạo nhiều URL. Quan hệ nullable cho phép guest tạo URL không cần đăng ký.
- **urls (1) - click_events (N)**: Một URL có thể có nhiều lượt click. CASCADE delete đảm bảo khi xóa URL thì các click event liên quan cũng bị xóa.
- **ON DELETE SET NULL ở urls.userId**: Khi xóa user, các URL của user đó không bị mất mà chỉ chuyển thành “orphan” (guest URL).

4.4. Redis Cache

Bên cạnh MySQL persistent storage, Redis được sử dụng làm cache layer với 2 mục đích chính:

Bảng 6: Hai tầng sử dụng Redis trong hệ thống

Mục đích	Key pattern	Value	TTL
Session store	<code>sess:<session-id></code>	JSON serialized session	7 ngày
URL cache	<code>url:<shortCode></code>	longUrl string	1 ngày

Ưu điểm của Redis cache:

- **Session store:** Stateless server, nhiều instance có thể chia sẻ session, không phụ thuộc bộ nhớ in-process.
- **URL cache:** Redirect cache hit < 5ms thay vì 10-20ms khi query MySQL. Với hệ thống URL Shortener mà 95%+ traffic là redirect, đây là tối ưu sống còn.

5. Backend

5.1. Khởi động server

`backend/src/server.js` là entry point đơn giản, làm nhiệm vụ load app và lắng nghe trên cổng cấu hình:

```

1 require('dotenv').config();
2 const app = require('./app');
3
4 const PORT = process.env.PORT || 3000;
5 app.listen(PORT, () => {
6   console.log('Server running on http://localhost:${PORT}');
7 });

```

Listing 2: File `backend/src/server.js`

`backend/src/app.js` thiết lập Express với pipeline middleware theo thứ tự. Thứ tự middleware quan trọng vì Express xử lý request theo thứ tự khai báo:

1. `trust proxy 1` - Tin tưởng 1 lớp proxy (Nginx) phía trước để lấy IP thực từ header `X-Forwarded-For`.
2. `cors` với `credentials: true` - Cho phép cookie cross-origin.
3. `express.json()` - Parse JSON body.
4. `cookieParser()` - Parse Cookie header.
5. `express-session` với custom `RedisStore` - Quản lý phiên.
6. Static files từ thư mục `frontend/`.
7. API routes: `/api/auth/*`, `/api/*`.
8. Redirect catch-all `/:shortCode` - Đặt sau cùng để không chặn API.
9. Global error handler - Đặt cuối cùng để bắt mọi lỗi.

5.2. Session Store tùy chỉnh

Thực hiện implement một `RedisStore` kế thừa từ `session.Store` để giảm dependency và minh họa rõ cơ chế hoạt động của session store:

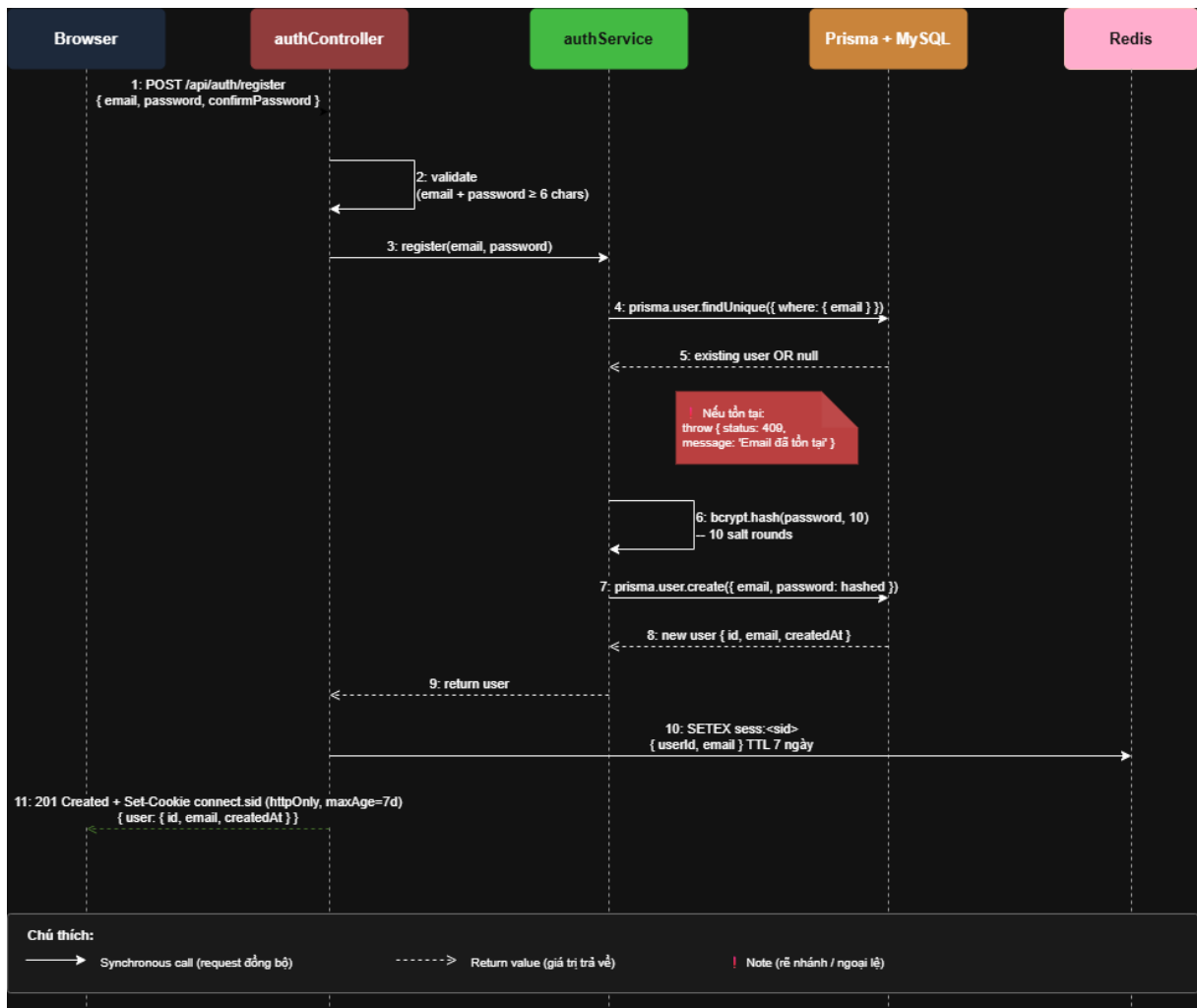
```
1 class RedisStore extends session.Store {
2   get(sid, cb) {
3     redis.get('sess:${sid}')
4       .then(data => cb(null, data ? JSON.parse(data) : null))
5       .catch(cb);
6   }
7   set(sid, sess, cb) {
8     const ttl = Math.floor((sess.cookie?.maxAge || 86400000) / 1000);
9     redis.setex('sess:${sid}', ttl, JSON.stringify(sess))
10      .then(() => cb(null)).catch(cb);
11   }
12   destroy(sid, cb) {
13     redis.del('sess:${sid}').then(() => cb(null)).catch(cb);
14   }
15 }
```

Listing 3: Custom RedisStore trong backend/src/config/session.js

Session key trong Redis: `sess:<session-id>`, TTL khớp với cookie `maxAge` (7 ngày).

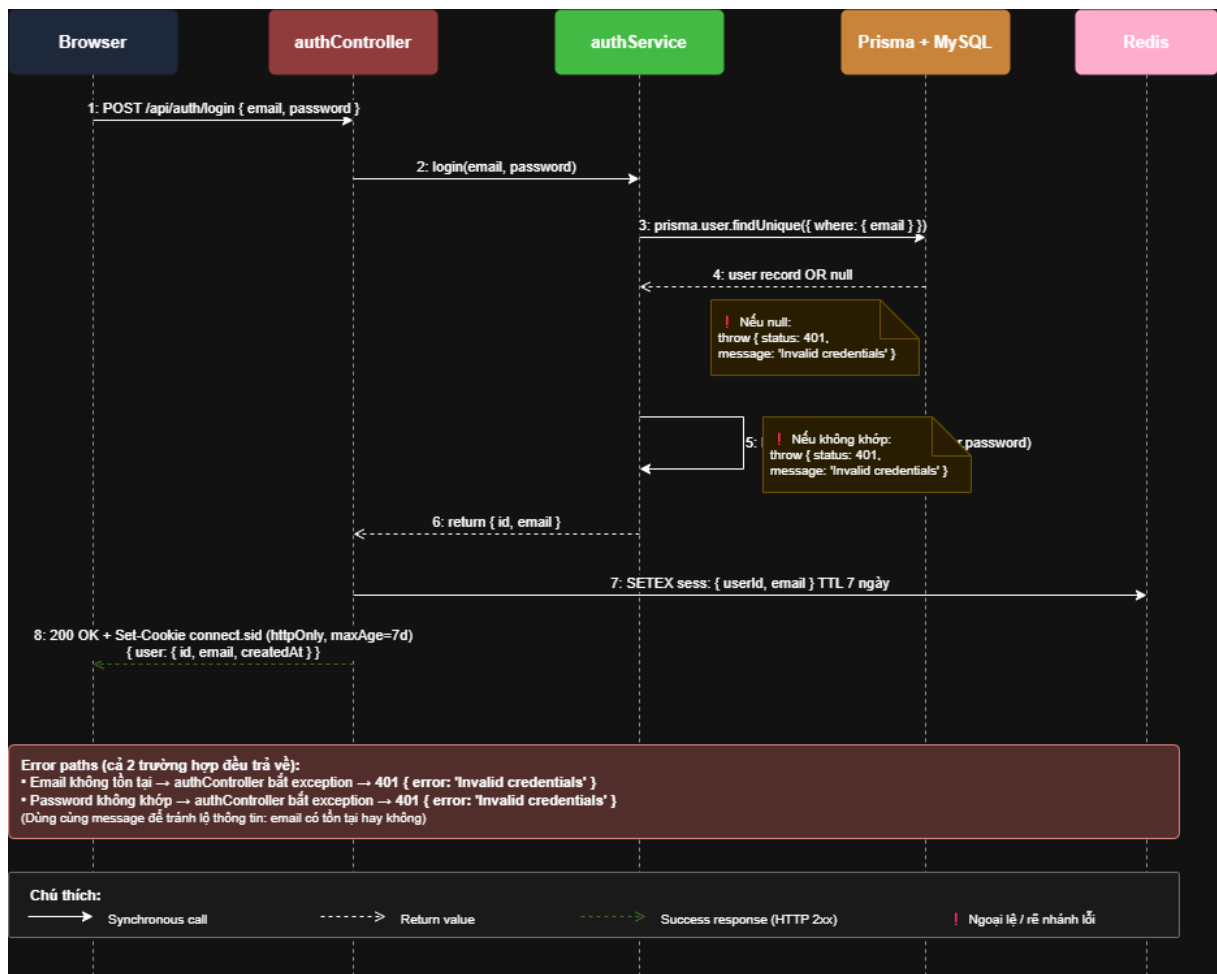
5.3. Luồng xác thực (Authentication Flow)

5.3.1. Luồng đăng ký (Register)



Hình 3: Luồng xử lý đăng ký tài khoản

5.3.2. Luồng đăng nhập (Login)



Hình 4: Luồng xử lý đăng nhập

5.3.3. Middleware bảo vệ route

```

1 // authMiddleware.js
2 function requireAuth(req, res, next) {
3   if (!req.session?.userId) {
4     return res.status(401).json({ error: 'Unauthorized' });
5   }
6   next();
7 }
8
9 function optionalAuth(req, res, next) {
10  // Không throw error nếu không có session
11  // Cho phép guest tạo URL nhưng vẫn track userId nếu đã login
12  next();
13 }
  
```

Listing 4: Middleware requireAuth

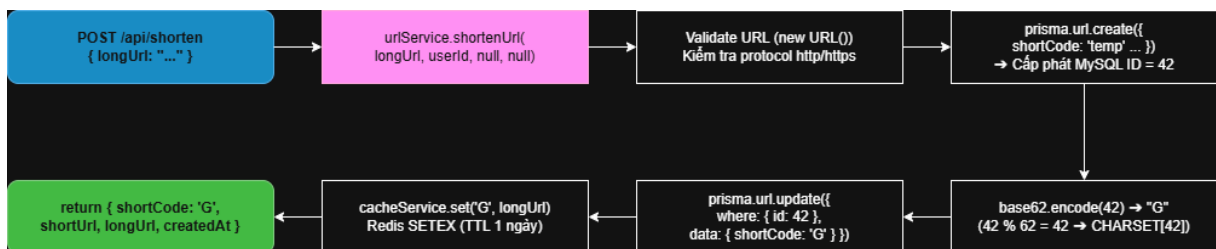
Hàm `requireAuth` kiểm tra `req.session.userId`. Nếu không có session hợp lệ (chưa login hoặc session đã hết hạn) -> trả về 401 Unauthorized.

5.4. URL Shortening - Logic hoạt động

`backend/src/services/urlService.js` chứa logic chính của tính năng rút gọn URL.

5.4.1. Trường hợp 1: Auto-generate shortCode (Base62)

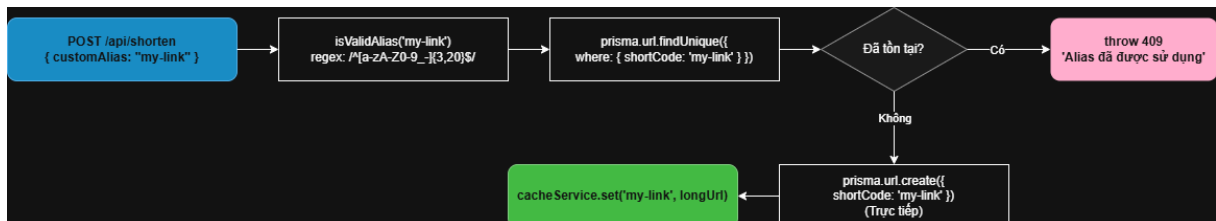
Là phương pháp tạo `shortCode` tự động, đảm bảo **zero collision** (không trùng lặp):



Hình 5: Luồng tạo shortCode bằng Base62

5.4.2. Trường hợp 2: Custom Alias

Người dùng có thể tự chỉ định mã rút gọn:



Hình 6: Luồng tạo URL với custom alias

5.4.3. Thuật toán Base62 Encoding

Base62 sử dụng bảng ký tự gồm 62 ký tự (10 chữ số + 26 chữ thường + 26 chữ hoa), mã hóa một số nguyên thành chuỗi ngắn.

```

1 const CHARSET = '0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ' +
2                 'abcdefghijklmnopqrstuvwxyz';
3 // 10 digits + 26 uppercase + 26 lowercase = 62 ký tự
4
5 function encode(num) {
6   if (num === 0) return '0';
7   let result = '';

```



```

8   while (num > 0) {
9       result = CHARSET[num % 62] + result; // lay phan du, gan vao dau
10      num = Math.floor(num / 62);          // tiep tục với phan
11      }                                     nguyen
12  }
13  return result;
14  }
15  function decode(str) {
16      let num = 0;
17      for (let i = 0; i < str.length; i++) {
18          num = num * 62 + CHARSET.indexOf(str[i]);
19      }
20      return num;
21  }
22
23  module.exports = { encode, decode };

```

Listing 5: File backend/src/utils/base62.js

Bảng 7: Tương quan giữa Auto-increment ID và Base62 shortCode

Auto-increment ID	Base62 shortCode	Số ký tự	Ý nghĩa
1	1	1	URL đầu tiên trong hệ thống
62	10	2	Sau 62 URL
3,844	100	3	62^2 URL
238,328	1000	4	62^3 URL
14,776,336	10000	5	62^4 URL
916,132,832	100000	6	62^5 URL
56,800,235,584	1000000	7	62^6 URL (≈ 56 tỷ URL)

Với 6 ký tự Base62, hệ thống có thể chứa hơn 56 tỷ URL duy nhất

5.5. Redirect - Cache-First Logic

- **Cache-first:** 95%+ request được phục vụ từ Redis ($< 5\text{ms}$) thay vì query MySQL (10-20ms).
- **Fire-and-forget:** Việc ghi `click_events` và `clicks++` được thực hiện không đồng bộ, không await. Nếu DB ghi chậm hoặc lỗi, user vẫn được redirect bình thường.

- **Warm cache on miss:** Khi cache miss, sau khi đọc từ MySQL, tự động set lại Redis để các request tiếp theo hit cache.

5.6. Analytics - Thu thập và tổng hợp

5.6.1. Ghi Click Event

```
1  const geoip = require('geoip-lite');
2
3  async function recordClick(urlId, ip, userAgent, referer) {
4    // Tra cứu quốc gia từ IP qua geoip-lite (offline DB)
5    let country = null;
6    if (ip) {
7      try {
8        const geo = geoip.lookup(ip);
9        if (geo && geo.country) country = geo.country; // ISO code: VN
10       , US, JP...
11     } catch (e) { /* ignore lookup errors */ }
12   }
13
14   await prisma.clickEvent.create({
15     data: {
16       urlId,
17       ipAddress: ip ? String(ip).substring(0, 45) : null,
18       userAgent: userAgent || null,
19       referer: referer || null,
20       country
21     }
22   });
23 }
```

Listing 6: Hàm ghi click event với GeoIP lookup

Ưu điểm của geoip-lite:

- **Offline lookup:** Database được đóng gói cùng package, không cần gọi API ngoài, đảm bảo latency < 1ms.
- **Không tốn chi phí:** Sử dụng GeoLite2 free database (cập nhật định kỳ).
- **Hỗ trợ IPv4 và IPv6:** Tra cứu chính xác cho cả hai loại địa chỉ.
- **Trả về ISO code 2 ký tự:** Tiết kiệm không gian lưu trữ trong DB (VARCHAR 50).

5.6.2. Tổng hợp Analytics 30 ngày

```
1  async function getAnalytics(urlId, days = 30) {
2    const since = new Date();
3    since.setDate(since.getDate() - days);
4  }
```

```
5   const events = await prisma.clickEvent.findMany({
6     where: { urlId, clickedAt: { gte: since } },
7     select: { clickedAt: true, referer: true, country: true }
8   });
9
10  // Khởi tạo bảng dữ liệu cho 30 ngày (đủ không có click vẫn hiển
    // thị 0)
11  const byDayMap = {};
12  for (let i = days - 1; i >= 0; i--) {
13    const d = new Date();
14    d.setDate(d.getDate() - i);
15    byDayMap[d.toISOString().split('T')[0]] = 0;
16  }
17
18  // Demo click theo ngày
19  events.forEach(e => {
20    const key = e.clickedAt.toISOString().split('T')[0];
21    if (byDayMap[key] !== undefined) byDayMap[key]++;
22  });
23
24  // Top 5 referers
25  const refererMap = {};
26  events.forEach(e => {
27    const r = e.referer || 'Direct';
28    refererMap[r] = (refererMap[r] || 0) + 1;
29  });
30  const topReferers = Object.entries(refererMap)
31    .sort((a, b) => b[1] - a[1])
32    .slice(0, 5)
33    .map(([referer, count]) => ({ referer, count }));
34
35  // Top countries (từ GeoIP lookup lúc recordClick)
36  const countryMap = {};
37  events.forEach(e => {
38    const c = e.country || 'Unknown';
39    countryMap[c] = (countryMap[c] || 0) + 1;
40  });
41  const topCountries = Object.entries(countryMap)
42    .sort((a, b) => b[1] - a[1])
43    .slice(0, 10)
44    .map(([country, count]) => ({ country, count }));
45
46  return {
47    totalClicks: events.length,
48    clicksByDay: byDayMap,
49    topReferers,
50    topCountries
```

```

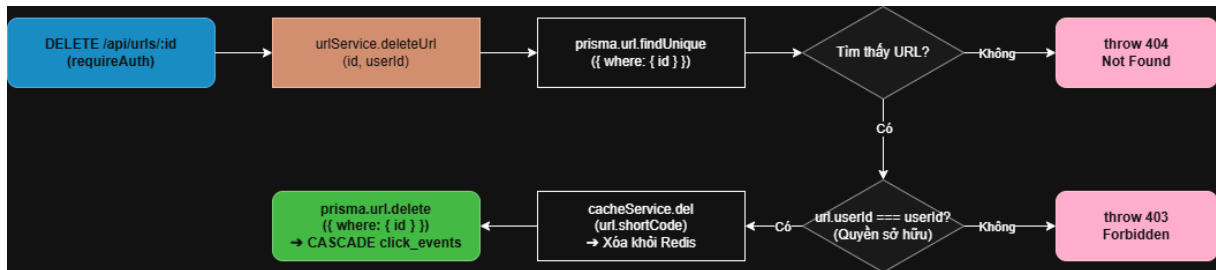
51   };
52 }

```

Listing 7: Hàm tổng hợp dữ liệu analytics

5.7. Dashboard - Quản lý URL

5.7.1. Luồng xóa URL



Hình 7: Luồng xóa URL với kiểm tra quyền sở hữu

5.8. Middleware bảo vệ hệ thống

5.8.1. Rate Limiter

backend/src/middleware/rateLimiter.js sử dụng express-rate-limit:

```

1  const rateLimit = require('express-rate-limit');
2
3  const shortenLimiter = rateLimit({
4    windowMs: 15 * 60 * 1000,    // 15 phút
5    max: 20,                    // 20 requests / 15 phút / IP
6    message: { error: 'Qua nhiều request, vui long thu lai sau' }
7  });
8
9  const authLimiter = rateLimit({
10   windowMs: 15 * 60 * 1000,
11   max: 10,
12   message: { error: 'Qua nhiều request, vui long thu lai sau' }
13 });
14
15 module.exports = { shortenLimiter, authLimiter };

```

Listing 8: Cấu hình Rate Limiter

Mục đích:

- Chống tấn công brute-force vào endpoint đăng nhập (10 req/15 phút).
- Hạn chế abuse tính năng rút gọn URL (20 req/15 phút).

5.8.2. Error Handler toàn cục

```
1 module.exports = (err, req, res, next) => {
2   console.error('[Error] ${err.message}');
3   const status = err.status || 500;
4   res.status(status).json({
5     error: err.message || 'Internal Server Error'
6   });
7 };
```

Listing 9: Global error handler

5.9. Health Check Endpoint

Endpoint dùng cho monitoring, kiểm tra trạng thái MySQL và Redis:

```
1 app.get('/api/health', async (req, res) => {
2   const status = { status: 'ok', uptime: process.uptime() };
3   try {
4     await prisma.$queryRaw`SELECT 1`;
5     status.db = 'ok';
6   } catch (e) { status.db = 'error'; }
7   try {
8     await redis.ping();
9     status.redis = 'ok';
10  } catch (e) { status.redis = 'error'; }
11  status.timestamp = new Date().toISOString();
12  res.json(status);
13 });
```

Listing 10: Health check endpoint

6. Frontend

6.1. Các trang giao diện

Bảng 8: Danh sách các trang giao diện

File	Chức năng
index.html	Trang chủ với form rút gọn URL
signin.html	Trang đăng nhập
register.html	Trang đăng ký
dashboard.html	Trang quản lý URL của người dùng
analytics.html	Trang hiển thị thống kê click bằng Chart.js
404.html	Trang lỗi khi không tìm thấy shortCode

6.2. Giao tiếp API - File api.js

frontend/js/api.js: triển khai lớp trừu tượng cho API `fetch()` để xử lý toàn bộ luồng trao đổi dữ liệu giữa frontend và backend:

```

1  var API_BASE = '/api';
2
3  // Hàm gốc wrapper cho fetch
4  async function apiCall(endpoint, options) {
5      var config = Object.assign({
6          headers: { 'Content-Type': 'application/json' },
7          credentials: 'include' // gửi session cookie cùng request
8      }, options || {});
9
10     var res = await fetch(API_BASE + endpoint, config);
11     var data = await res.json();
12
13     if (!res.ok) {
14         var err = new Error(data.error || 'Error ' + res.status);
15         err.status = res.status;
16         throw err;
17     }
18     return data;
19 }
20
21 // Export 2 namespace tiện dụng
22 window.AuthApi = {
23     register: (email, pwd) => apiCall('/auth/register', {

```

```

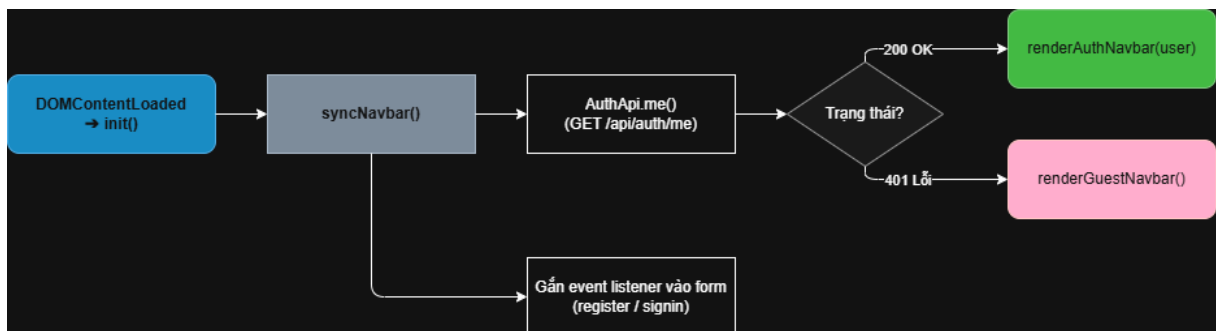
24     method: 'POST', body: JSON.stringify({ email, password: pwd })
25   }),
26   login: (email, pwd) => apiCall('/auth/login', {
27     method: 'POST', body: JSON.stringify({ email, password: pwd })
28   }),
29   logout: () => apiCall('/auth/logout', { method: 'POST' }),
30   me: () => apiCall('/auth/me')
31 };
32
33 window.ShortenApi = {
34   shorten: (longUrl, alias, expiresAt) => apiCall('/shorten', {
35     method: 'POST',
36     body: JSON.stringify({ longUrl, customAlias: alias, expiresAt })
37   })
38 };

```

Listing 11: Wrapper API trong api.js

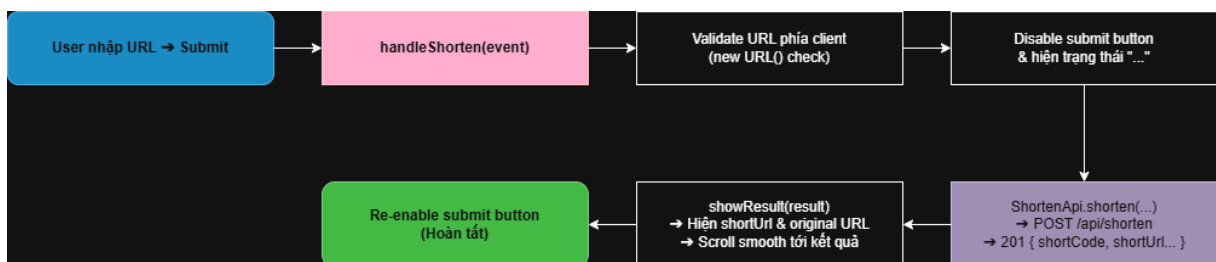
- credentials: 'include' - Gửi kèm cookie session để xác thực phiên.
- Wrap lỗi thành Error object có status - giúp caller xử lý theo HTTP code.

6.3. Xác thực và đồng bộ Navbar - auth.js



Hình 8: Luồng đồng bộ trạng thái đăng nhập trên Navbar

6.4. Form rút gọn URL - shorten.js

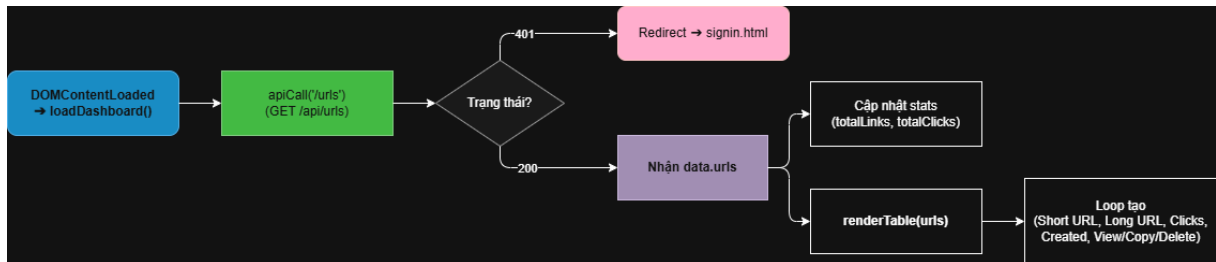


Hình 9: Luồng rút gọn URL từ phía client

Tính năng **copy to clipboard** có 2 nhánh fallback:

1. Nếu browser hỗ trợ `navigator.clipboard` (HTTPS only) -> dùng API hiện đại.
2. Fallback: tạo `<textarea>` ẩn, `select()` và `document.execCommand('copy')`.

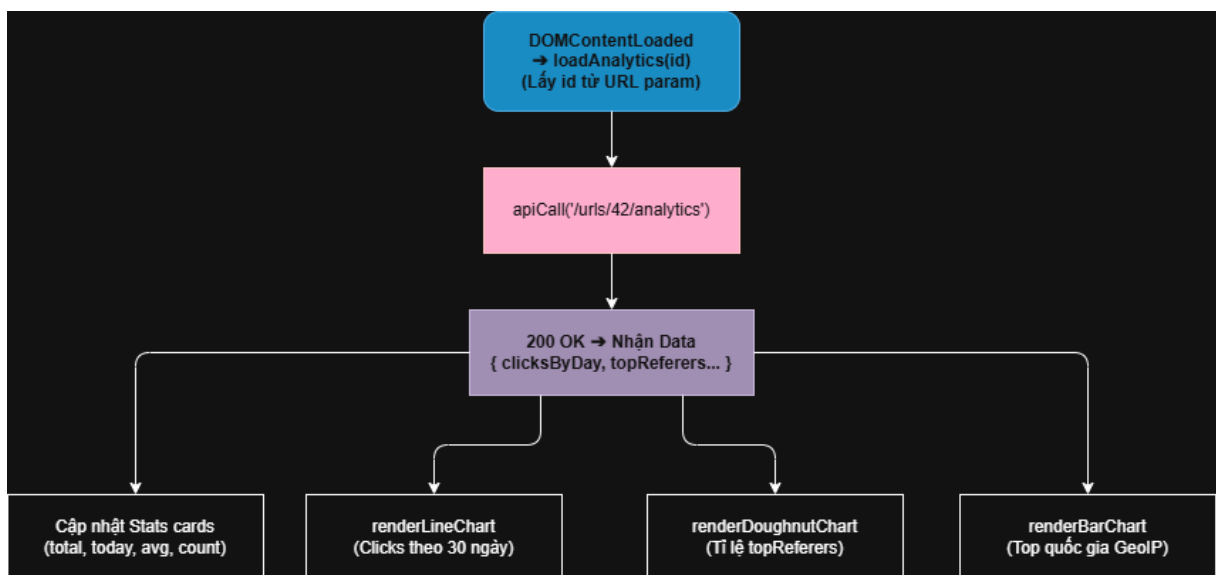
6.5. Dashboard - dashboard.js



Hình 10: Luồng tải Dashboard

6.6. Trang Analytics - analytics.js

Hiển thị 3 loại biểu đồ với Chart.js:



Hình 11: Luồng tải trang Analytics

6.7. Utility functions - utils.js

```

1 // XSS prevention: escape HTML trước khi innerHTML
2 function escapeHtml(str) {
3   if (!str) return '';
4   return String(str)
5     .replace(/&/g, '&amp;')
6     .replace(/</g, '&lt;');
  
```



```

7      .replace(/>/g, '&gt;')
8      .replace(/"/g, '&quot;')
9      .replace(/'/g, '&#039;');
10   }
11
12   // Format date theo dd/MM/yyyy HH:mm
13   function formatDate(isoString) {
14       const d = new Date(isoString);
15       return d.toLocaleString('vi-VN');
16   }
17
18   // Truncate long URL
19   function truncate(str, maxlen) {
20       if (str.length <= maxlen) return str;
21       return str.substring(0, maxlen - 3) + '...';
22   }

```

Listing 12: Hàm tiện ích chung

7. API Endpoints

7.1. Bảng tổng hợp API

Bảng 9: Bảng tổng hợp các API endpoint của hệ thống

Method	Endpoint	Auth	Rate Limit	Mô tả
POST	/api/auth/register	-	10/15min	Đăng ký, set session
POST	/api/auth/login	-	10/15min	Đăng nhập, set session
POST	/api/auth/logout	-	-	Destroy session
GET	/api/auth/me	-	-	Lấy user hiện tại
POST	/api/shorten	Optional	20/15min	Tạo short URL
GET	/:shortCode	-	-	Redirect 302
GET	/api/urls	Required	-	List URL của user
DELETE	/api/urls/:id	Required	-	Xóa URL (ownership check)
PATCH	/api/urls/:id	Required	-	Cập nhật expiry
GET	/api/urls/:id/analytics	Required	-	Analytics data
GET	/api/health	-	-	Trạng thái DB + Redis

7.2. Chi tiết Request/Response các API chính

7.2.1. POST /api/auth/register

Request:

```
1 POST /api/auth/register
2 Content-Type: application/json
3
4 {
5   "email": "user@example.com",
6   "password": "securepass123",
7   "confirmPassword": "securepass123"
8 }
```

Listing 13: Request body /api/auth/register

Response (201 Created):

```
1 HTTP/1.1 201 Created
2 Set-Cookie: connect.sid=s%3A...; Path=/; HttpOnly; SameSite=Lax
3
4 {
5   "user": {
6     "id": 5,
7     "email": "user@example.com",
8     "createdAt": "2026-05-14T10:30:00.000Z"
9   }
10 }
```

Listing 14: Response /api/auth/register thành công

Response lỗi (409 Conflict):

```
1 { "error": "Email đã được đăng ký" }
```

7.2.2. POST /api/shorten

Request:

```
1 POST /api/shorten
2 Content-Type: application/json
3 Cookie: connect.sid=... (optional cho guest)
4
5 {
6   "longUrl": "https://example.com/very/long/path?param=value",
7   "customAlias": "my-link",           // optional
8   "expiresAt": "2026-12-31T23:59:59Z" // optional
9 }
```

Listing 15: Request body /api/shorten

Response (201 Created):

```
1 {
2   "shortCode": "my-link",
3   "shortUrl": "https://sht-url.me/my-link",
4   "longUrl": "https://example.com/very/long/path?param=value",
5   "createdAt": "2026-05-14T10:35:00.000Z",
6   "expiresAt": "2026-12-31T23:59:59.000Z"
7 }
```

Listing 16: Response /api/shorten thành công**7.2.3. GET /:shortCode (Redirect)****Request:**

```
1 GET /my-link
```

Response:

```
1 HTTP/1.1 302 Found
2 Location: https://example.com/very/long/path?param=value
```

Các trường hợp lỗi:

- **404 Not Found:** ShortCode không tồn tại -> sendFile 404.html.
- **410 Gone:** URL đã hết hạn (`expiresAt < now`).

7.2.4. GET /api/urls/:id/analytics**Response:**

```
1 {
2   "url": {
3     "id": 42,
4     "shortCode": "my-link",
5     "longUrl": "https://example.com/...",
6     "createdAt": "2026-05-01T10:00:00Z"
7   },
8   "totalClicks": 145,
9   "clicksByDay": {
10    "2026-04-15": 5,
11    "2026-04-16": 12,
12    "2026-04-17": 8,
13    "...": "..."
14  },
15  "topReferers": [
16    { "referer": "https://twitter.com", "count": 45 },
17    { "referer": "Direct", "count": 30 },
18    { "referer": "https://facebook.com", "count": 25 }
```

```

19   ],
20   "topCountries": [
21     { "country": "VN", "count": 78 },
22     { "country": "US", "count": 32 },
23     { "country": "JP", "count": 15 },
24     { "country": "SG", "count": 12 },
25     { "country": "Unknown", "count": 8 }
26   ]
27 }

```

Listing 17: Response analytics 30 ngày

7.3. HTTP Status Code

Bảng 10: HTTP Status Code được sử dụng

Code	Tên	Trường hợp sử dụng
200	OK	GET thành công, login thành công
201	Created	POST tạo mới thành công (register, shorten)
302	Found	Redirect từ shortCode sang longUrl
400	Bad Request	Validation lỗi (URL không hợp lệ, alias sai format)
401	Unauthorized	Chưa đăng nhập, sai mật khẩu
403	Forbidden	Đăng nhập rồi nhưng không phải chủ sở hữu URL
404	Not Found	ShortCode không tồn tại, URL ID không có
409	Conflict	Email đã đăng ký, alias đã được sử dụng
410	Gone	URL đã hết hạn
429	Too Many Requests	Vượt quá rate limit
500	Internal Server Error	Lỗi không xác định

8. Redis

8.1. Tổng quan kiến trúc Redis

Bảng 11: Cấu hình Redis trong hệ thống

Mục đích	Key pattern	Value	TTL
Session store	<code>sess:<session-id></code>	JSON serialized session	7 ngày
URL cache	<code>url:<shortCode></code>	longUrl string	1 ngày

8.2. Tầng 1: Session Store

8.2.1. Vai trò

Lưu trữ thông tin phiên đăng nhập của người dùng. Khi user đăng nhập, server tạo một session record gồm:

- **userId:** ID của user.
- **email:** Email để hiển thị nhanh.
- **cookie:** Cấu hình cookie (maxAge, expires).

Session được serialize thành JSON và lưu vào Redis với key `sess:<session-id>`. Server trả về cookie `connect.sid` chứa session ID đã ký HMAC.

8.2.2. Lưu đồ hoạt động



Hình 12: Luồng hoạt động của Session Store

8.2.3. Ưu điểm của Redis làm Session Store

- **Scalable:** Khi mở rộng nhiều instance app, tất cả đều chia sẻ chung Redis -> session hoạt động liền mạch.
- **Persistent:** Redis có RDB snapshot, session không mất khi restart container.
- **Fast:** Latency < 1ms, không ảnh hưởng đến performance request.
- **TTL tự động:** Khi cookie hết hạn, Redis tự xóa session record.

8.3. Tầng 2: URL Cache

8.3.1. Vai trò

Cache mapping `shortCode` -> `longUrl` để tăng tốc redirect. Đây là tối ưu sống còn vì 95%+ traffic của hệ thống URL Shortener là redirect.

8.3.2. Cache Service Wrapper

```

1  const redis = require('../config/redis');
2  const TTL = 24 * 60 * 60; // 1 ngày (giay)
3
4  module.exports = {
5    async get(key) {
6      try {
7        return await redis.get('url:${key}');
8      } catch (e) {
9        console.error('Cache get error:', e.message);
10       return null; // Fail-safe: tra null neu Redis loi
11     }
12   },
13   async set(key, val, ttl = TTL) {
14     try {
15       await redis.setex('url:${key}', ttl, val);
16     } catch (e) {
17       console.error('Cache set error:', e.message);
18     }
19   },
20   async del(key) {
21     try {
22       await redis.del('url:${key}');
23     } catch (e) {
24       console.error('Cache del error:', e.message);
25     }
26   }
27 };

```

Listing 18: File backend/src/services/cacheService.js

8.3.3. Chiến lược Cache Invalidation

Bảng 12: Chiến lược quản lý cache

Sự kiện	Hành động
Tạo URL mới	SET cache ngay khi INSERT thành công
Redirect (cache miss)	Sau khi đọc từ MySQL, SET cache (warm up)
Cập nhật URL (PATCH)	DELETE cache cũ, request tiếp theo sẽ warm lại
Xóa URL (DELETE)	DELETE cache đồng thời với DELETE từ MySQL
Tự động	TTL hết hạn -> Redis tự xóa

8.4. So sánh hiệu năng với và không có cache

Bảng 13: So sánh latency redirect

Trường hợp	Latency trung bình	Giải thích
Cache hit (Redis)	2-5 ms	GET Redis local + redirect
Cache miss (MySQL)	10-25 ms	Query MySQL + SET Redis + redirect
Không có Redis	10-25 ms mỗi request	Mọi request đều query MySQL

Với traffic 1,000 req/s và 95% cache hit rate, hệ thống giảm tải MySQL từ 1,000 query/s xuống còn 50 query/s - giúp DB rảnh rỗi để phục vụ các request phức tạp hơn (analytics, dashboard).

8.5. Cấu hình ioredis

```

1  const Redis = require('ioredis');
2
3  const redis = new Redis({
4    host: process.env.REDIS_HOST || 'localhost',
5    port: parseInt(process.env.REDIS_PORT) || 6379,
6    retryStrategy: (times) => Math.min(times * 50, 2000),
7    maxRetriesPerRequest: 3
8  });
9
10 redis.on('connect', () => console.log('[Redis] Connected'));
11 redis.on('error', (err) => console.error('[Redis] Error:', err.
    message));
12
13 module.exports = redis;
```

Listing 19: File backend/src/config/redis.js

- **retryStrategy:** Tự động retry khi mất kết nối với backoff tăng dần.
- **maxRetriesPerRequest:** Giới hạn retry để tránh hang request.

9. Đóng gói và triển khai với Docker

9.1. Tổng quan Container hóa

Việc đóng gói ứng dụng bằng Docker giúp:

- **Tính nhất quán:** Môi trường dev = test = production.
- **Cô lập (Isolation):** Mỗi service (app, mysql, redis, nginx) chạy trong container riêng.
- **Triển khai một lệnh:** `docker compose up -build` khởi động toàn bộ stack.
- **Dễ nhân bản:** Sao chép cấu hình lên VPS khác chỉ cần copy `docker-compose.yml` và `.env`.

9.2. Dockerfile của ứng dụng

```
1 # Stage 1: Build dependencies
2 FROM node:18-alpine AS builder
3 WORKDIR /app
4
5 # Cài đặt thư viện hệ thống cần thiết cho Prisma trên Alpine
6 RUN apk add --no-cache openssl libc6-compat
7
8 COPY backend/package*.json ./
9 RUN npm ci --only=production
10
11 COPY backend/prisma ./prisma
12 RUN npx prisma generate
13
14 # Stage 2: Runtime
15 FROM node:18-alpine
16 WORKDIR /app
17
18 RUN apk add --no-cache openssl libc6-compat
19
20 COPY --from=builder /app/node_modules ./node_modules
21 COPY --from=builder /app/prisma ./prisma
22 COPY backend/src ./src
23 COPY frontend ../frontend
24
25 ENV NODE_ENV=production
26 EXPOSE 3000
27
28 CMD ["node", "src/server.js"]
```

Listing 20: File Dockerfile của backend

9.3. Docker Compose Configuration

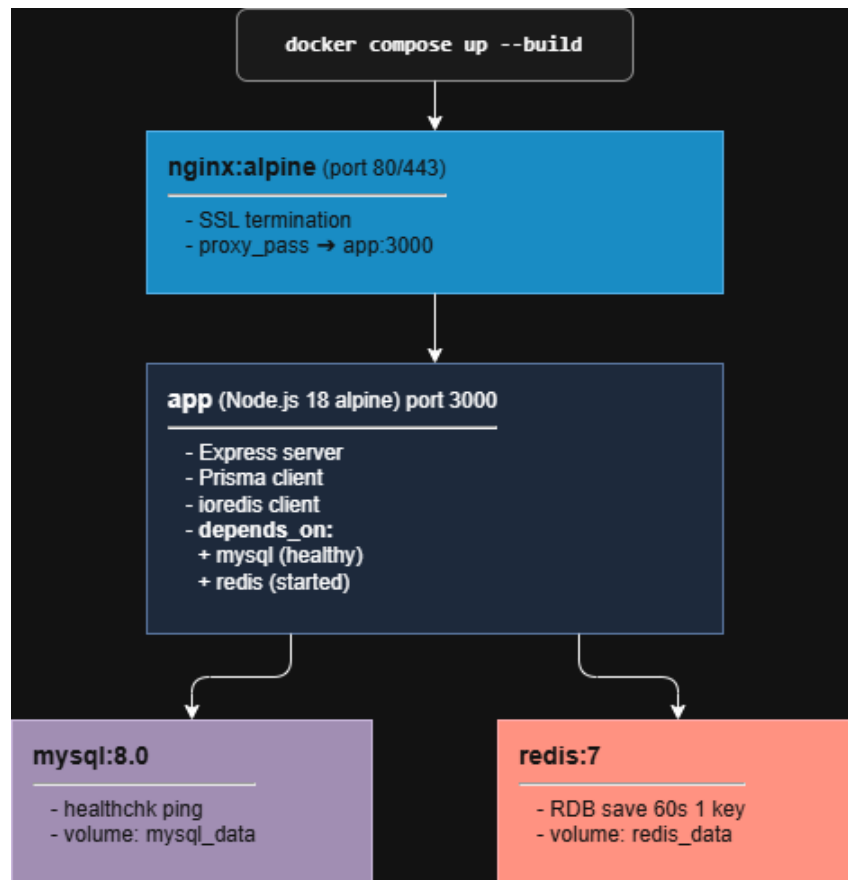
```
1 version: '3.8'
2
3 services:
4   app:
```

```
5     build:
6         context: .
7         dockerfile: Dockerfile
8     container_name: url_shortener_app
9     restart: unless-stopped
10    ports:
11        - "3000:3000"
12    environment:
13        DATABASE_URL: mysql://root:${DB_PASSWORD}@mysql:3306/
url_shortener
14        REDIS_HOST: redis
15        REDIS_PORT: 6379
16        SESSION_SECRET: ${SESSION_SECRET}
17        APP_URL: ${APP_URL}
18        NODE_ENV: production
19    depends_on:
20        mysql:
21            condition: service_healthy
22        redis:
23            condition: service_started
24    networks:
25        - app_network
26
27    mysql:
28        image: mysql:8.0
29        container_name: url_shortener_mysql
30        restart: unless-stopped
31        environment:
32            MYSQL_ROOT_PASSWORD: ${DB_PASSWORD}
33            MYSQL_DATABASE: url_shortener
34        volumes:
35            - mysql_data:/var/lib/mysql
36        healthcheck:
37            test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
38            interval: 10s
39            timeout: 5s
40            retries: 10
41        networks:
42            - app_network
43
44    redis:
45        image: redis:7-alpine
46        container_name: url_shortener_redis
47        restart: unless-stopped
48        command: redis-server --save 60 1 --loglevel warning
49        volumes:
50            - redis_data:/data
```

```
51     networks:
52         - app_network
53
54     nginx:
55         image: nginx:alpine
56         container_name: url_shortener_nginx
57         restart: unless-stopped
58         ports:
59             - "80:80"
60             - "443:443"
61         volumes:
62             - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
63             - /etc/letsencrypt:/etc/letsencrypt:ro
64         depends_on:
65             - app
66         networks:
67             - app_network
68
69     volumes:
70         mysql_data:
71         redis_data:
72
73     networks:
74         app_network:
75             driver: bridge
```

Listing 21: File docker-compose.yml

9.4. Sơ đồ kiến trúc Docker



Hình 13: Kiến trúc các container Docker

- **Volume persistent:** `mysql_data` và `redis_data` đảm bảo dữ liệu không mất khi restart container.
- **Healthcheck MySQL:** App chỉ start khi MySQL đã sẵn sàng (`condition: service_healthy`).
- **Redis RDB snapshot:** `-save 60 1` - tạo snapshot mỗi 60 giây nếu có ≥ 1 key thay đổi.
- **Restart policy:** `unless-stopped` - tự khởi động lại khi container crash hoặc reboot host.
- **Network bridge:** Các container giao tiếp qua tên service (`mysql`, `redis`) thay vì IP cứng.

10. Bảo mật

10.1. Tổng quan các biện pháp bảo mật

Bảng 14: Bảng tổng hợp các biện pháp bảo mật

Biện pháp	Cách hiện thực
Password hashing	<code>bcrypt.hash(password, 10)</code> - 10 salt rounds
Session cookie	<code>httpOnly: true</code> - không đọc được qua JS
Session secret	<code>SESSION_SECRET</code> từ env var (đủ dài, ngẫu nhiên)
CSRF protection	<code>sameSite: 'lax'</code> trên cookie
Secure cookie (HTTPS)	<code>secure: true</code> khi <code>NODE_ENV=production</code>
Ownership check	Mọi route <code>/api/urls/:id</code> kiểm tra <code>url.userId === session.userId</code>
Rate limiting	10 req/15min auth, 20 req/15min shorten
Input validation	<code>isValidUrl()</code> , <code>isValidAlias()</code> trước khi INSERT
XSS prevention	<code>escapeHtml()</code> ở frontend trước khi <code>innerHTML</code>
SQL injection	Prisma parameterized queries - không raw SQL với user input
IP spoofing	<code>trust proxy 1</code> kết hợp Nginx X-Forwarded-For
HTTPS	Let's Encrypt SSL, Nginx redirect HTTP -> HTTPS
HSTS	Header <code>Strict-Transport-Security</code> qua Nginx

10.2. Chi tiết một số biện pháp quan trọng

10.2.1. Password Hashing với Bcrypt

```

1  const bcrypt = require('bcryptjs');
2
3  // Khi register
4  const hashed = await bcrypt.hash(password, 10); // 10 salt rounds
5  await prisma.user.create({ email, password: hashed });
6
7  // Khi login
8  const ok = await bcrypt.compare(password, user.password);
9  if (!ok) throw { status: 401, message: 'Invalid credentials' };

```

Listing 22: Hash và verify password**10.2.2. Ownership Check (Authorization)**

Mọi mutation endpoint trên `/api/urls/:id` đều kiểm tra quyền sở hữu:

```
1 async function deleteUrl(id, userId) {
2   const url = await prisma.url.findUnique({ where: { id } });
3   if (!url) throw { status: 404, message: 'Not found' };
4
5   // kiểm tra ownership
6   if (url.userId !== userId) {
7     throw { status: 403, message: 'Forbidden' };
8   }
9
10  await cacheService.del(url.shortCode);
11  await prisma.url.delete({ where: { id } });
12 }
```

Listing 23: Ownership check

Là cơ chế chống **IDOR (Insecure Direct Object Reference)** - user A không thể xóa/sửa URL của user B chỉ bằng cách đoán ID.

10.2.3. XSS Prevention

Khi render danh sách URL ở frontend, mọi giá trị từ DB đều được escape:

```
1 function escapeHtml(str) {
2   return String(str)
3     .replace(/&/g, '&amp;')
4     .replace(/</g, '&lt;')
5     .replace(/>/g, '&gt;')
6     .replace(/"/g, '&quot;')
7     .replace(/'/g, '&#039;');
8 }
9
10 // Su dung
11 tdLongUrl.innerHTML = escapeHtml(url.longUrl);
```

Listing 24: Escape HTML chống XSS

Nếu attacker tạo URL với `longUrl = "<script>alert(1)</script>"`, output sẽ là text plain, không thực thi script.

10.2.4. SQL Injection Prevention

Prisma sử dụng parameterized queries mặc định. Mọi truy vấn dạng `prisma.url.findUnique({ where: { shortCode } })` đều an toàn vì giá trị được truyền qua parameter, không nối chuỗi SQL.

10.3. Bảo mật cookie

```

1 app.use(session({
2   store: new RedisStore(),
3   secret: process.env.SESSION_SECRET,
4   resave: false,
5   saveUninitialized: false,
6   cookie: {
7     httpOnly: true, // chống XSS doc
7     cookie
8     secure: process.env.NODE_ENV === 'production', // HTTPS only
9     sameSite: 'lax', // chống CSRF
10    maxAge: 7 * 24 * 60 * 60 * 1000 // 7 ngày
11  }
12 }));

```

Listing 25: Cấu hình session cookie an toàn

11. Triển khai Production và HTTPS

11.1. Hạ tầng Máy chủ (VPS)

- **Nhà cung cấp:** DigitalOcean (Region Singapore - SGP1).
- **Cấu hình:** Ubuntu 24.04 LTS, 1 vCPU / 1GB RAM / 25GB SSD.
- **Địa chỉ IP:** 168.144.106.190.
- **Bảo mật truy cập:** Vô hiệu hóa đăng nhập mật khẩu, sử dụng **SSH Keys** để ngăn chặn tấn công Brute-force. Kết nối qua: `ssh root@168.144.106.190`.

11.2. Quy trình thiết lập Docker và xử lý sự cố

11.2.1. Cài đặt môi trường

```

1 curl -fsSL https://get.docker.com -o get-docker.sh && sh get-docker.
   sh
2 docker --version
3 docker compose version

```

Listing 26: Lệnh cài đặt Docker official script

11.2.2. Xử lý lỗi thư viện Prisma (OpenSSL)

Sự cố: Khi chạy trên nền Alpine Linux trong Docker, Prisma Query Engine báo lỗi thiếu thư viện hệ thống:

```
1 PrismaClientInitializationError:
2 Unable to require libquery_engine-linux-musl.so.node
```

Giải pháp: Bổ sung vào Dockerfile:

```
1 RUN apk add --no-cache openssl libc6-compat
```

11.2.3. Cấu hình phục vụ giao diện tĩnh

Sửa đổi `server.js` để Express phục vụ đúng các file trong thư mục `frontend/` khi triển khai container:

```
1 app.use(express.static(path.join(__dirname, '../..../frontend')));
```

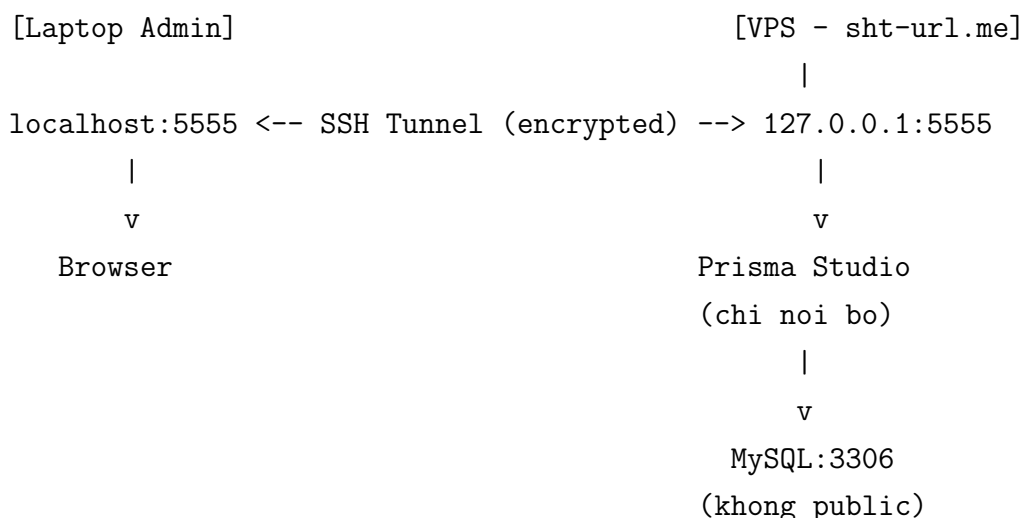
11.3. Quản trị Database an toàn (SSH Tunneling)

Để bảo mật dữ liệu, cổng MySQL (3306) **không được mở công khai** ra Internet. Thay vào đó, sử dụng cơ chế đường hầm mã hóa:

1. **Server-side:** Chạy Prisma Studio giới hạn tại 127.0.0.1:5555 bên trong mạng nội bộ Docker.
2. **Client-side:** Tạo SSH Tunnel từ máy cá nhân:

```
1 ssh -L 5555:127.0.0.1:5555 root@168.144.106.190
```

3. **Kết quả:** Admin truy cập `http://localhost:5555` để quản trị DB qua kết nối đã được mã hóa bởi SSH.



Hình 14: Sơ đồ SSH Tunneling cho quản trị DB

11.4. Cấu hình tên miền và HTTPS

11.4.1. Đăng ký tên miền

- Tên miền: sht-url.me.
- Nguồn: Đăng ký miễn phí qua GitHub Student Developer Pack (Namecheap).
- Cấu hình DNS: A Record trỏ về 168.144.106.190.

11.4.2. Xử lý xung đột Port 80

Khi cài đặt Nginx trên VPS đã chạy Docker, gặp lỗi port 80 bị chiếm:

```
1 ss -tulpn | grep :80
2 # tcp LISTEN 0 4096 0.0.0.0:80 docker-proxy
```

Giải pháp: Stop docker-proxy đang chiếm port, để Nginx trong container đảm nhiệm:

```
1 docker compose stop nginx      # neu co
2 sudo systemctl stop nginx      # neu cai nginx system
3 # Su dung nginx trong docker-compose.yml
```

11.4.3. Cài đặt SSL với Let's Encrypt

```
1 # Cai dat Certbot
2 sudo apt install certbot python3-certbot-nginx
3
4 # Cap chung chi
5 sudo certbot --nginx -d sht-url.me -d www.sht-url.me
6
7 # Tu dong renew (cron job)
8 sudo certbot renew --dry-run
```

Listing 27: Cài đặt SSL miễn phí với Certbot

11.4.4. Cấu hình Nginx Reverse Proxy

```
1 # Redirect HTTP -> HTTPS
2 server {
3     listen 80;
4     server_name sht-url.me www.sht-url.me;
5     return 301 https://$host$request_uri;
6 }
7
8 # HTTPS server
9 server {
```

```

10     listen 443 ssl http2;
11     server_name sht-url.me www.sht-url.me;
12
13     ssl_certificate /etc/letsencrypt/live/sht-url.me/fullchain.pem;
14     ssl_certificate_key /etc/letsencrypt/live/sht-url.me/privkey.pem;
15     ssl_protocols TLSv1.2 TLSv1.3;
16     ssl_ciphers HIGH:!aNULL:!MD5;
17
18     # Security headers
19     add_header Strict-Transport-Security
20             "max-age=31536000; includeSubDomains" always;
21     add_header X-Frame-Options DENY;
22     add_header X-Content-Type-Options nosniff;
23     add_header X-XSS-Protection "1; mode=block";
24
25     location / {
26         proxy_pass http://app:3000;
27         proxy_http_version 1.1;
28         proxy_set_header Host $host;
29         proxy_set_header X-Real-IP $remote_addr;
30         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
31         proxy_set_header X-Forwarded-Proto $scheme;
32     }
33 }

```

Listing 28: File nginx/default.conf

11.5. Quy trình deploy

```

1  # 1. SSH vào VPS
2  ssh root@168.144.106.190
3
4  # 2. Clone hoặc pull code mới nhất
5  cd /opt/url-shortener
6  git pull origin main
7
8  # 3. Cập nhật .env (nếu cần)
9  nano .env
10
11 # 4. Build và khởi động lại
12 docker compose down
13 docker compose up -d --build
14
15 # 5. Chạy migration nếu schema thay đổi
16 docker compose exec app npx prisma migrate deploy
17

```

```
18 # 6. Kiểm tra status
19 docker compose ps
20 docker compose logs -f app
```

Listing 29: Quy trình deploy lên Production

12. Kết luận

12.1. Kết luận

Đồ án **URL Shortener** hoàn thành các tính năng cơ bản cần thiết, và hoàn thiện phát triển phần mềm từ thiết kế, code, test đến triển khai Production thực tế với tên miền và HTTPS.

Hệ thống được xây dựng theo: kiến trúc phân lớp, ORM type-safe, cache-first redirect, container hóa, reverse proxy, SSL/TLS encryption. Điều này giúp em có cái nhìn thực tế và toàn diện về quy trình phát triển một ứng dụng web hoàn chỉnh, từ ý tưởng đến sản phẩm chạy thật trên Internet.

12.2. Hạn chế của đồ án

- **GeoIP database miễn phí:** Đồ án sử dụng `geoip-lite` với GeoLite2 free database, độ chính xác ở cấp quốc gia (chưa có city-level). Để nâng cấp cần MaxMind GeoIP2 trả phí.
- **Chưa có OAuth:** Chỉ hỗ trợ đăng ký email/password, chưa có Google/Facebook login.
- **Chưa có CI/CD:** Deploy vẫn thủ công qua SSH, chưa có pipeline tự động với GitHub Actions.
- **Chưa có monitoring:** Chỉ có endpoint `/api/health` cơ bản, chưa tích hợp Prometheus + Grafana.
- **Single instance:** Chưa horizontal scaling, chỉ chạy 1 instance app.

Tài liệu tham khảo

1. Express.js Documentation. <https://expressjs.com/>
2. Prisma ORM Documentation. <https://www.prisma.io/docs>
3. Redis Official Documentation. <https://redis.io/docs/>
4. MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/refman/8.0/en/>
5. Docker Documentation. <https://docs.docker.com/>
6. Nginx Reverse Proxy Guide. <https://docs.nginx.com/nginx/admin-guide/web-server/reverse-proxy/>
7. Let's Encrypt - Certbot. <https://certbot.eff.org/>
8. Bcrypt - Password Hashing. <https://github.com/kelektiv/node.bcrypt.js>
9. Chart.js Documentation. <https://www.chartjs.org/docs/>
10. OWASP Top 10 Web Application Security Risks. <https://owasp.org/www-project-top-ten/>
11. Base62 Encoding for URL Shortening. <https://en.wikipedia.org/wiki/Base62>
12. Bitly Engineering Blog - System Design. <https://bitly.com/blog/>